



UNIVERSIDADE DO ESTADO DO AMAZONAS
ESCOLA SUPERIOR DE TECNOLOGIA
NÚCLEO DE COMPUTAÇÃO - NUCOMP

ALGORITMOS E ESTRUTURAS DE DADOS I – 2026/1
PROF. FLÁVIO COELHO

PROJETO PRÁTICO 1 (versão 3)

1 Objetivos

Este **Projeto Prático 1 – PP1** tem o objetivo de avaliar sua habilidade em codificar em C++20 os Tipos Abstratos de Dados **Lista**, **Pilha** e **Fila** (em suas implementações estáticas e/ou dinâmicas) conforme mostrado nas aulas de AED 1 e nas referências indicadas pelo professor, com o objetivo de resolver um problema de média complexidade que pode incluir/combina todos os assuntos já abordados em AED 1. Além disso, será avaliada a sua habilidade em desenvolver uma solução correta para o problema proposto neste enunciado, atendendo à todos os requisitos pedidos.

2 Descrição do problema



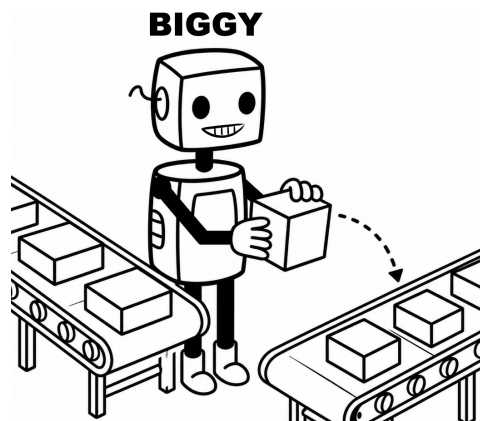
A **Grande Irmão Varejo Interestelar Corporation** ou apenas **GIVIC**, é a mais gigantesca, descomunal, colossal e indecentemente enorme rede de varejo da galáxia! Seu império está tendo problemas com um punhado de rebeldes e você precisa de um laser desintegrador de planetas? A GIVIC te entrega! Anseia degustar um porção das apetitosas lesmas radioativas do Cinturão de Kuiper? A GIVIC te entrega! E aquele clone robô para lhe substituir no

amigo oculto da empresa? A GIVIC te entrega! Não importa em que planeta você vive, em que nebulosa você se esconde! A GIVIC te entrega, pequeno irmão e irmã! Porém, parece que um dos queridos estagiários da GIVIC, conhecido pelo nome de Bob Wilson, em um momento de pura e inocente distração, apagou todo o código do sistema de processamento de entregas! E para coroar a sua destreza em criar desastres monumentais, Bob Wilson também deletou, inocentemente, todas as cópias de backup do sistema. Então, você e sua equipe foram recrutados para resolver o problema e reconstruir todo o sistema de processamento de entregas da GIVIC do zero! Bom, o antigo time de desenvolvimento usava a famosa linguagem de programação alienígena $\text{?}\bullet\text{--}\bullet\text{--}\text{?}\text{+}\text{+}\text{+}$ (que significa: “Todos que usarem essa linguagem fracassarão miseravelmente!”). Por motivos desconhecidos, o uso de $\text{?}\bullet\text{--}\bullet\text{--}\text{?}\text{+}\text{+}\text{+}$ não foi bem sucedido. Então, o pessoal da GIVIC lhe pediu para codificar o novo sistema em C++20.

Como funciona o sistema de processamento de entregas da GIVIC?

O processamento de entregas da GIVIC é realizado com a ajuda de dois robôs: Biggy e Bang. Biggy encaminha os pacotes de pedidos de clientes para o embarque no transporte de entregas e Bang recebe esses pacotes encaminhados por Biggy e os carrega no transporte apropriadamente.

A tarefa de Biggy



Biggy fica diante de uma esteira esperando a chegada de pacotes de pedidos de clientes. Antes dos pacotes chegarem, o sistema de logística da GIVIC envia quatro estruturas de dados para Biggy:

- (a) **A fila de comandos** do Biggy (Biggy-COMANDOS): contém comandos que, ao serem lidos pelo Biggy, fazem com que ele execute ações específicas sobre os pacotes de pedidos passando pela esteira;
- (b) **A fila de embarque de pacotes** (FEP): esta fila inicia vazia e vai sendo preenchida por Biggy com alguns dos comandos que ele recebe da fila Biggy-COMANDOS, ou seja, Biggy repassa alguns comandos da Biggy-COMANDOS para a FEP;
- (c) **A fila de pacotes cancelados** (CANCEL): esta lista inicia vazia e vai sendo preenchida por Biggy com pacotes cuja entrega foi cancelada;

(d) A **fila de descarte** (DESC): esta lista inicia vazia e vai sendo preenchida por Biggy com pacotes que devem ser descartados pois sofreram alguma avaria.

Quando os pacotes começam a chegar, Biggy lê o próximo comando da Biggy-COMANDOS e começa a executá-lo. Cada comando faz Biggy executar uma ação específica sobre o pacote que está passando pela esteira. Veja como Biggy atua quando recebe certos comandos:

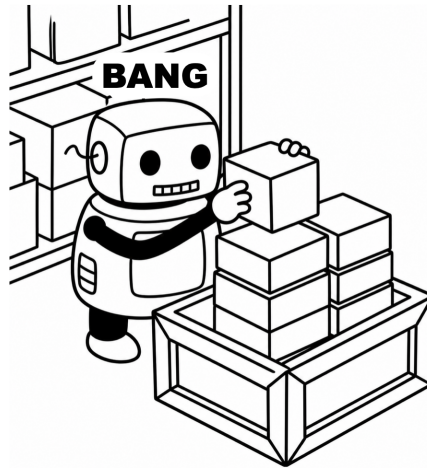
- Quando Biggy recebe, por exemplo, um comando ('E', 789), significa que o pacote 789 (que está diante dele na esteira) está pronto para ser entregue. Então, Biggy move o pacote dessa esteira para a esteira de embarque de pacotes. Ao mesmo tempo, Biggy enfileira o comando ('E', 789) na FEP. A FEP será utilizada depois pelo seu amigo Bang.
- Se Biggy recebe um comando ('C', 789), por exemplo, significa que a entrega do pacote 789 foi cancelada. Nesse caso, Biggy verifica se o pacote 789 já foi incluído na FEP. Em caso positivo, Biggy não pode mais fazer nada pois o pacote físico já está à caminho de ser carregado no transporte de entregas. Ainda assim, Biggy enfileira o comando ('C', 789) na FEP para ser tratado depois pelo Bang. Caso o pacote 789 ainda não tenha sido movido para a FEP (ou seja, não seja encontrado na FEP), Biggy transfere o pacote físico para uma esteira de pacotes cancelados. Em seguida inclui o identificado do pacote (o número 789) na fila CANCEL. Posteriormente, Bang também manipulará a fila CANCEL.
- Se Biggy, por exemplo, retira um comando ('A', 789), significa que o pacote está com alguma avaria física e deve ser descartado. Assim como ocorre no cancelamento, Biggy verifica se o pacote 789 já foi incluído na FEP. Em caso positivo, Biggy apenas enfileira o comando ('A', 789) na FEP para, depois, ser tratado pelo Bang. Caso o pacote 789 ainda não tenha sido movido para a FEP, Biggy o transfere para uma esteira de descarte de pacotes. Em seguida, Biggy inclui o identificado do pacote (o número 789) na fila DESC. Posteriormente, Bang também manipulará a fila CANCEL.
- Caso Biggy retira o comando $(-, 0)$, ele repassa o comando para a FEP e entra em modo de espera, pois concluiu sua tarefa.

Veja um exemplo de ações do Biggy na Tabela 1 e como fica a FEP, a CANCEL e a DESC, ao final da tarefa.

Comando	Ação de Biggy na memória	FEP	CANCEL	DESC
	Recebe estruturas de dados	[]	[]	[]
E 24	Repassa o comando para a FEP	[(E, 24)]	[]	[]
E 80	Repassa o comando para a FEP	[(E, 24), (E, 80)]	[]	[]
C 24	24 já está na esteira de embarque (não tem como cancelar agora); repassa o comando para a FEP	[(E, 24), (E, 80), (C, 24)]	[]	[]
E 105	Repassa o comando para a FEP	[(E, 24), (E, 80), (C, 24), (E, 105)]	[]	[]
E 11	Repassa o comando para a FEP	[(E, 24), (E, 80), (C, 24), (E, 105), (E, 11)]	[]	[]
E 86	Repassa o comando para a FEP	[(E, 24), (E, 80), (C, 24), (E, 105), (E, 11), (E, 86)]	[]	[]
C 30	Move 30 para CANCEL	[(E, 24), (E, 80), (C, 24), (E, 105), (E, 11), (E, 86)]	[30]	[]
A 86	86 já está na esteira de embarque (não tem como descartar agora); repassa o comando para a FEP	[(E, 24), (E, 80), (C, 24), (E, 105), (E, 11), (E, 86), (A, 86)]	[30]	[]
A 33	Move 33 para DESC	[(E, 24), (E, 80), (C, 24), (E, 105), (E, 11), (E, 86), (A, 86)]	[30]	[33]
A 105	105 já está na esteira de embarque (não tem como descartar agora); repassa o comando para a FEP	[(E, 24), (E, 80), (C, 24), (E, 105), (E, 11), (E, 86), (A, 86), (A, 105)]	[30]	[33]
- 0	Repassa o comando para a FEP e entra em modo de espera	[(E, 24), (E, 80), (C, 24), (E, 105), (E, 11), (E, 86), (A, 86), (A, 105), (-, 0)]	[30]	[33]

Tabela 1: Exemplo de ações de Biggy.

A tarefa de Bang



Quando Biggy conclui sua tarefa, é a vez de seu amigo Bang entrar em ação! O sistema de logística da GIVIC repassa para Bang as filas FEP, CANCEL e DESC preenchidas anteriormente por Biggy. Ao executar os comandos da FEP, um após o outro, Bang começa a desempenhar a sua tarefa que é carregar os pacotes da esteira de embarque de pacotes para a **pilha de entrega de pacotes** (PEP) que fica localizada dentro da nave de transporte de entregas. Para realizar essa tarefa, Bang mantém uma versão digital da PEP em sua memória, além da FEP, CANCEL e DESC. Veja como Bang reage a cada comando:

- Quando Bang retira, por exemplo, um comando ('E', 456) da FEP, significa que o pacote 456 (que está diante dele na esteira) está pronto para ser embarcado. Então, Bang move o pacote dessa esteira para a PEP física dentro da nave de transporte. Ao mesmo tempo, Bang empilha o identificador 456 na PEP em sua memória.
- Se, por exemplo, Bang retira um comando ('C', 456) da FEP, significa que a entrega do pacote 456 foi cancelada. Nesse caso, o pacote 456 que já foi empilhado na PEP física, precisa ser retirado de lá e colocado na esteira de pacotes cancelados. Então, em sua memória, Bang transfere cada identificador de pacote da PEP para uma pilha auxiliar AUX, até desempilhar o identificador do pacote 456. Nesse momento, Bang enfileira 456 na fila CANCEL. Após isso, Bang transfere de volta os identificadores de AUX para a PEP até esvaziar a AUX. Isso conclui sua ação. Enquanto executa esse procedimento em sua memória, no mundo real, Bang realiza o mesmo procedimento entre a PEP física e uma pilha auxiliar física, e ao final, move o pacote 456 para a esteira de pacotes cancelados.
- Se Bang retira um comando ('A', 456) da FEP, por exemplo, significa que o pacote 456 que já foi empilhado na PEP está com alguma avaria física e deve ser descartado. Assim como ocorre no procedimento de cancelamento (acima), Bang irá transferir os itens da PEP para a pilha AUX até desempilhar o identificador 456 e incluí-lo na DESC. Para concluir o procedimento, Bang devolverá os itens de AUX para a PEP. O mesmo ocorre no mundo real com as PEP física e a pilha auxiliar física. Ao final, o pacote recuperado e avariado é retirado da PEP física e é enfileirado na esteira de descartados.

- Caso Bang retire da FEP o comando $(-, 0)$, ele entra em modo de espera, pois concluiu sua tarefa.

Veja um exemplo de ações do Bang na Tabela 2 e como fica a PEP, a CANCEL e a DESC, ao final da tarefa (note que a FEP não é mostrada na Tabela 2, porque cada comando que Bang retira da FEP já é mostrado na coluna **Comando**).

Comando	Ação de Bang na memória	PEP	AUX	CANCEL	DESC
	Recebe estruturas de dados	[]	[]	[30]	[33]
E 24	Empilha 24 na PEP	[24]	[]	[30]	[33]
E 80	Empilha 80 na PEP	[80, 24]	[]	[30]	[33]
C 24	PEP $\Rightarrow$ AUX	24, []	[80]	[30]	[33]
	Enfileira 24 em CANCEL	[]	[80]	[30, 24]	[33]
	PEP $\Leftarrow$ AUX	[80]	[]	[30, 24]	[33]
E 105	Empilha 105 na PEP	[105, 80]	[]	[30, 24]	[33]
E 11	Empilha 11 na PEP	[11, 105, 80]	[]	[30, 24]	[33]
E 86	Empilha 86 na PEP	[86, 11, 105, 80]	[]	[30, 24]	[33]
A 86	Não processa PEP $\Rightarrow$ AUX completo porque 86 é achado no topo de PEP	86, [11, 105, 80]	[]	[30, 24]	[33]
	Enfileira 86 em DESC	[11, 105, 80]	[]	[30, 24]	[33, 86]
	Não processa PEP $\Leftarrow$ AUX, pois AUX está vazia	[11, 105, 80]	[]	[30, 24]	[33, 86]
A 105	PEP $\Rightarrow$ AUX	105, [80]	[11]	[30, 24]	[33, 86]
	Enfileira 105 em DESC	[80]	[11]	[30, 24]	[33, 86, 105]
	PEP $\Leftarrow$ AUX	[11, 80]	[]	[30, 24]	[33, 86, 105]
- 0	Entra em modo de espera	[11, 80]	[]	[30, 24]	[33, 86, 105]

Tabela 2: Exemplo de ações de Bang.

O que sua equipe deve fazer

Você e sua equipe precisam implementar todo o comportamento que ocorre nas memórias dos robôs Biggy e Bang, utilizando as estruturas de dados **fila** e **pilha** (e **lista**, se você precisar), com implementação por alocação dinâmica e generalizadas (template). A pilha e a fila devem ser implementadas preferencialmente a partir da implementação da estrutura de dados **deque** passada como exercício. Veja os diagramas da fila (queue) e da pilha (stack) nas Figuras 1 e 2. Implemente o comportamento de cada robô separadamente, ou seja, utilize uma função ou classe para cada um deles (claro, além dessas funções ou classes para os robôs você pode criar quantas funções/classes auxiliares precisar).

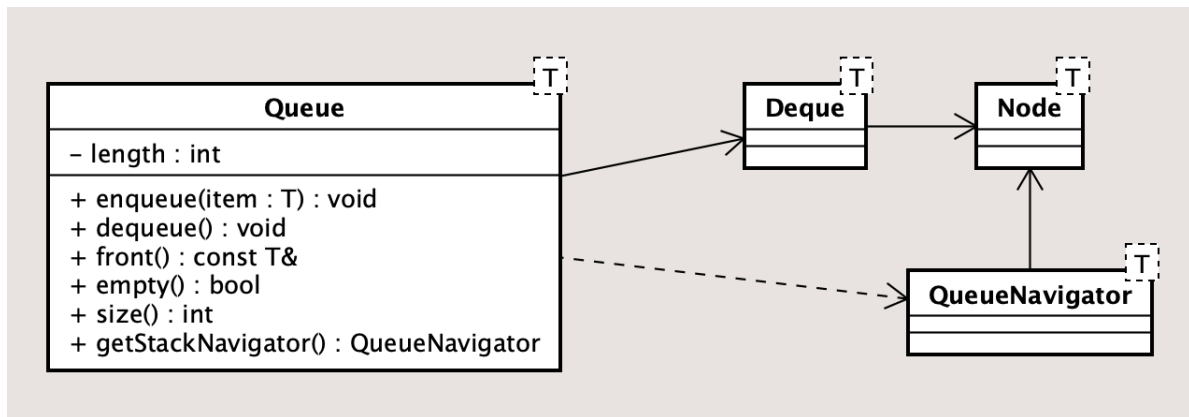


Figura 1: Classe Queue implementada com um Deque.

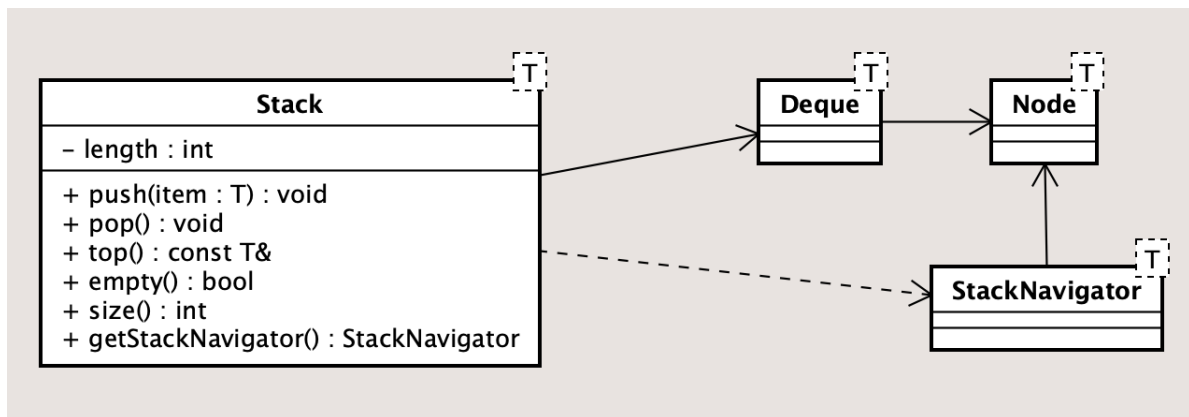


Figura 2: Classe Stack implementada com um Deque.

Formato de entradas e saídas

Seu programa deve ler a entrada do juiz on-line e processar/mostrar a saída correta. O formato de uma entrada de dados e de sua saída correspondente é mostrado no exemplo abaixo.

Entrada:

```

E 24
E 80
C 24
E 105
E 11
E 86
C 30
A 86
A 33
A 105
- 0
  
```

Saída:

Biggy:

```
FEP = [(E, 24), (E, 80), (C, 24), (E, 105), (E, 11), (E, 86), (A, 86), (A, 105), (-, 0)]
```

```
CANCEL = [30]
```

```
DESC = [33]
```

Bang:

```
FEP = [ ]
```

```
PEP = [11, 80]
```

```
AUX = [ ]
```

```
CANCEL = [30, 24]
```

```
DESC = [33, 86, 105]
```

3 Requisitos do projeto

1. **Equipes.** Este projeto deve ser desenvolvido por duplas ou individualmente. **Não serão aceitas equipes com mais de dois participantes com exceção daquelas permitidas pelo professor por razões justificáveis.**
2. **Ferramentas e técnicas.** O projeto deve ser codificado em C++20. Nenhum contêiner da STL (`<stack>`, `<queue>`, `<deque>`, `<vector>`, `<list>`, etc) é permitido. O uso de qualquer outra biblioteca não é permitido, com exceção de `<iostream>`, `<format>`, `<cmath>`, `<string>`, `<cctype>`, `<iomanip>`, `<limits>` e `<optional>`. Pelo menos os TADs fila, pilha, deque e lista devem ser codificados com programação orientada a objetos (classe, encapsulamento e templates). No restante do código, você pode usar POO ou programação procedural simples conforme sua escolha. Como compilador local, sugiro o GCC no Linux (<https://gcc.gnu.org/>), MinGW no Windows (<https://www.mingw-w64.org/>) ou o Clang no MacOS (<https://clang.llvm.org/>). Como editor/IDE local, sugiro o <https://www.sublimetext.com/> ou <https://code.visualstudio.com/> combinado com o compilador local C++20 sugerido acima. Para codificação online, sugiro <https://cpp.sh/> (marque o C++20) ou <https://onecompiler.com>.
3. **Correção e pontuação.** A correção e pontuação do PP1 é dividida em duas etapas:

Correção funcional - CF. Essa correção é realizada de forma automática pelo juiz on-line ao qual a equipe submeterá o seu código do PP1. Essa correção tem pontuação máxima igual a 10,0 pts. Apenas **um aluno fixo da equipe** deve submeter o código ao juiz on-line, quantas vezes quiser até a data e hora do prazo final de entrega. Ao submeter o código, o juiz on-line executará N casos de teste para testar a corretude da solução submetida. Portanto, pontuação da solução submetida corresponde ao número de casos de teste corretos, ou seja, $10,0/N * C$, onde N é o número de casos de teste do juiz on-line e C é o número de casos de teste corretos. Os detalhes sobre a submissão serão repassados pelo professor, por e-mail ou pelo Classroom.

Correção de Inspeção de código - IC. Essa correção é realizada manualmente pelo professor, que examinará a última versão do código submetido ao juiz on-line para

verificar se esse código atende aos requisitos deste enunciado. Em caso positivo, o professor confirma os pontos obtidos na CF. Em caso contrário, se um ou mais requisitos não forem seguidos, a pontuação obtida na CF sofrerá penalizações que reduzem o valor obtido na CF. Isso é necessário porque uma solução pode obter a nota máxima nas submissões ao juiz on-line, mas não atender a um ou mais requisitos solicitados neste enunciado. Por exemplo, um requisito pode pedir a implementação de uma algoritmo ou estrutura de dados X e, como solução, a equipe implementa um algoritmo ou estrutura de dados Y, diferente do que foi pedido. Dessa forma, qualquer solução submetida que não atenda um ou mais dos requisitos da Tabela 3 recebe uma penalidade (acumulativa) sobre a CF para cada requisito não atendido.

Tipo	Requisito	Penalidade por não cumprir
AED	Codifique uma classe C++20 fila (queue) com implementação dinâmica (de preferência com um deque interno), conforme as referências, sem usar estruturas de dados prontas de qualquer outra biblioteca	-20% do CF
AED	Codifique uma classe C++20 pilha (stack) com implementação dinâmica (de preferência com um deque interno), conforme as referências, sem usar estruturas de dados prontas de qualquer outra biblioteca	-20% do CF
AED	Codifique em C++20 a lógica separada do robô Biggy (dentro de uma função ou de uma classe própria)	-10% do CF
AED	Codifique em C++20 a lógica separada do robô Bang (dentro de uma função ou de uma classe própria)	-10% do CF
Linguagem	Utilize a linguagem C++20 em todo o projeto	-100% do CF
	Use corretamente funções e seus parâmetros (ponteiros, constantes, referências, etc), templates e classes C++; não utilize variáveis globais	-10% do CF
	Não use herança e/ou classes friend	-30% do CF
Estilo de código	Use indentação no padrão K&R ou Allman (veja https://en.wikipedia.org/wiki/Indentation_style) e nomeação de variáveis com Camel case ou Snake case ou Pascal case (veja https://en.wikipedia.org/wiki/Naming_convention_(programming))	-10% do CF
	Use um mesmo padrão de estilo de nomeação de identificadores e garanta que eles expressam bem os conceitos do problema (veja https://t.ly/2owrY e pule direto para “P.1: Express ideas directly in code”)	-10% do CF

Tabela 3: Requisitos do PP1 que devem ser atendidos